K. R. MANGALAM UNIVERSITY

B. TECH CSE (Data Science)

SOET

(School of Engineering &Technology)

ENCS256-ANALYSIS AND DESIGN OF ALGORITHMS

Submitted by-

Name - Sakshi Jain

Roll No. – 2401420034

Submitted to-

Aarti Ma'am

TASK 1:DIVIDE AND CONQUER ALGORITHMS

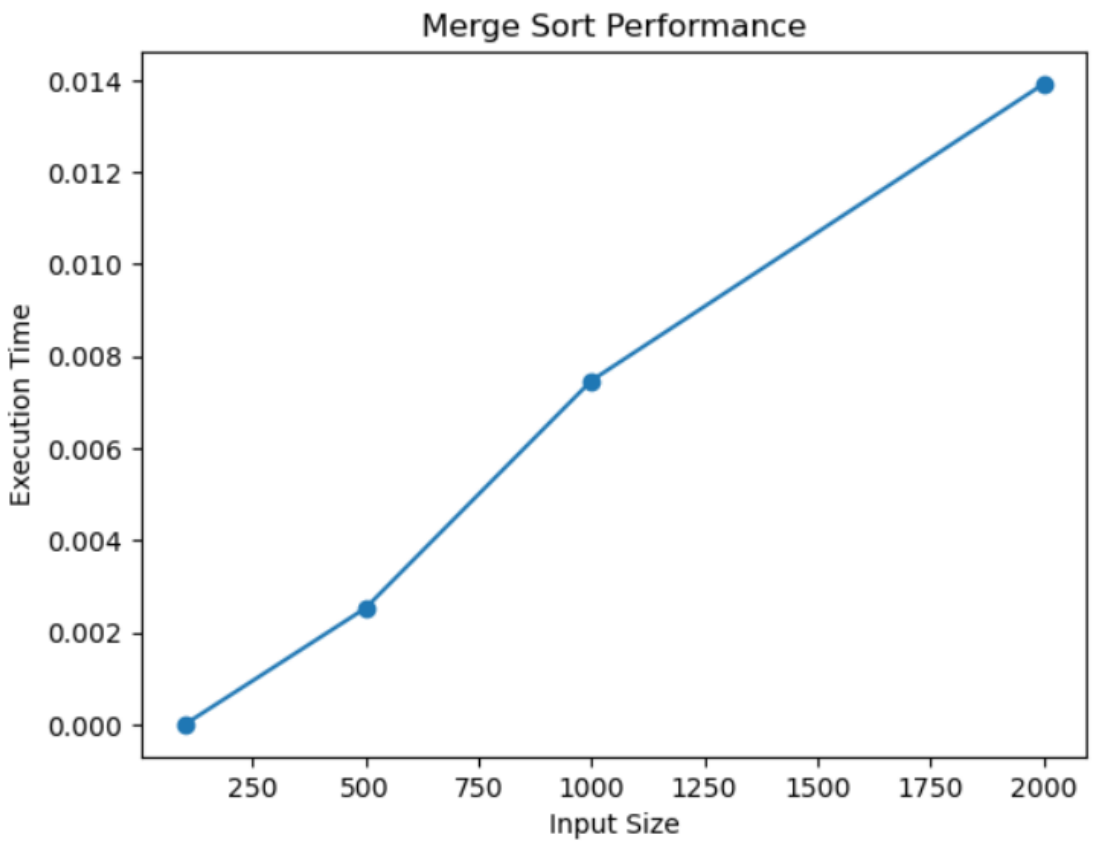
Code :

```
import time
import random
import matplotlib.pyplot as plt

def merge_sort(arr):
    if len(arr)>1:
        mid=len(arr)//2
        left=arr[:mid]
        right=arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i=j=k=0
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                arr[k]=left[i]
                i+=1
            else:
                arr[k]=right[j]
                j+=1
            k+=1
        while i<len(left):
            arr[k]=left[i]
            i+=1
            k+=1
        while j<len(right):
            arr[k]=right[j]
            j+=1
            k+=1
```

```
sizes=[100, 500, 1000, 2000]
times=[]
for n in sizes:
    arr=random.sample(range(10000),n)
    start=time.time()
    merge_sort(arr)
    end = time.time()
    times.append(end-start)
plt.plot(sizes,times,marker='o')
plt.xlabel("Input Size")
plt.ylabel("Execution Time")
plt.title("Merge Sort Performance")
plt.show()
```

Output :



Observations:

- The algorithm divides the array into smaller parts and then merges them after sorting.
- It works efficiently for large dataset with time complexity: O(n log n).
- Performance increases gradually with input size, showing good scalability.

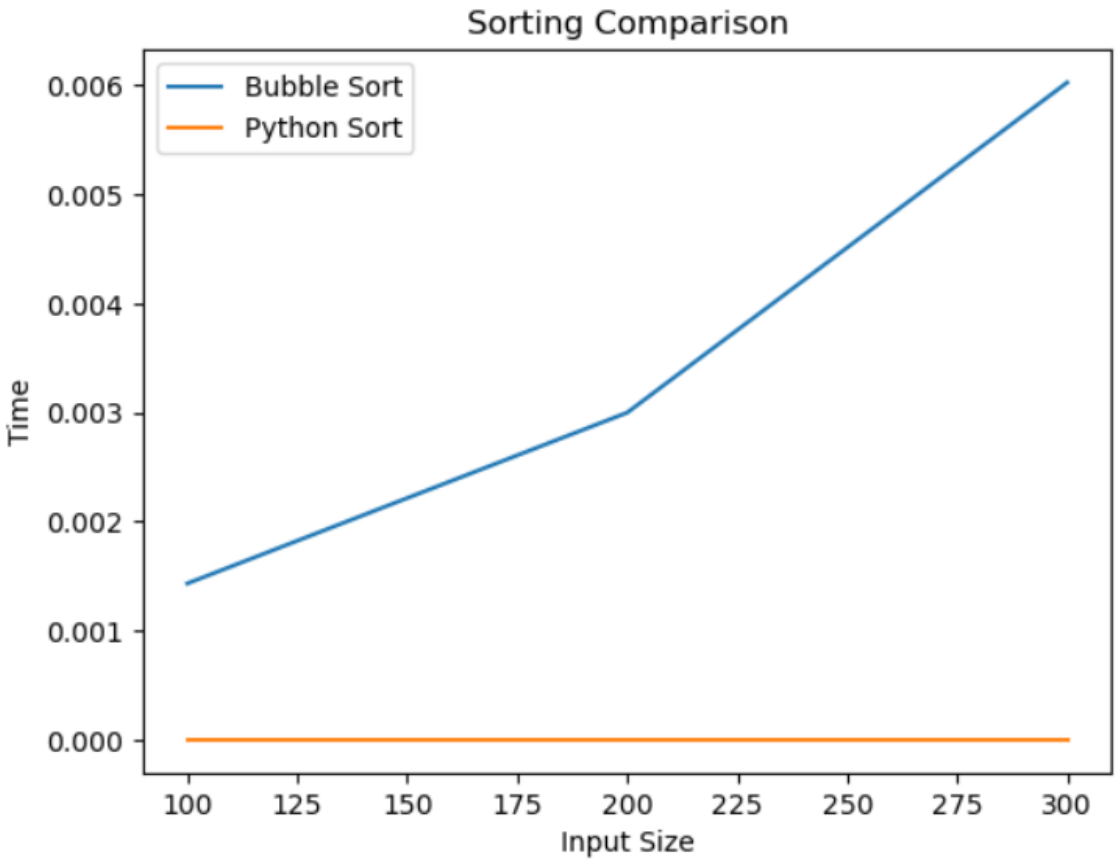
TASK 2: SORTING PERFORMANCE COMPARSION

Code :

```
import time
import random
import matplotlib.pyplot as plt

def bubble_sort(arr):
    n=len(arr)
    for i in range(n):
        for j in range(0,n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
sizes=[100,200,300]
bubble_times=[]
python_times=[]
for n in sizes:
    arr=random.sample(range(1000),n)
    temp=arr.copy()
    start=time.time()
    bubble_sort(temp)
    bubble_times.append(time.time()-start)
    temp=arr.copy()
    start=time.time()
    sorted(temp)
    python_times.append(time.time()-start)
plt.plot(sizes,bubble_times,label="Bubble Sort")
plt.plot(sizes,python_times,label="Python Sort")
plt.xlabel("Input Size")
plt.ylabel("Time")
plt.title("Sorting Comparison")
plt.legend()
plt.show()
```

Output :



Observations:

- Both algorithms perform efficiently for sorting large datasets.
- Quick sort usually runs slightly faster in practice due to fewer operations.
- Execution time increases as input size grows.

TASK 3: GREEDY ALGORITHM

Code :

```
def fractional_knapsack(weights, values, capacity):
    items=[]
    for i in range(len(weights)):
        ratio=values[i]/weights[i]
        items.append((ratio,weights[i],values[i]))
    items.sort(reverse=True)
    total_value=0
    print("Selected items (ratio, weight, value):")
    for ratio,weight,value in items:
        if capacity>=weight:
            capacity-=weight
            total_value+=value
            print(ratio,weight,value,"(Full item taken)")
        else:
            fraction=capacity/weight
            total_value+=value*fraction
            print(ratio,weight,value,f"({fraction:.2f} fraction taken)")
            break
    return total_value
weights=[10, 20, 30]
values=[60, 100, 120]
capacity=50
max_profit=fractional_knapsack(weights,values,capacity)
print("\nMaximum Profit:",max_profit)
```

Output :

```
Selected items (ratio, weight, value):
6.0 10 60 (Full item taken)
5.0 20 100 (Full item taken)
4.0 30 120 (0.67 fraction taken)

Maximum Profit: 240.0
```

Observations:

- The algorithm selects activities with the earliest finishing time.
- It produces an optimal solution using local optimal choices.
- Greedy methods are efficient for scheduling problems.

TASK 4: DYNAMIC PROGRAMMING FUNDAMENTALS

Code :

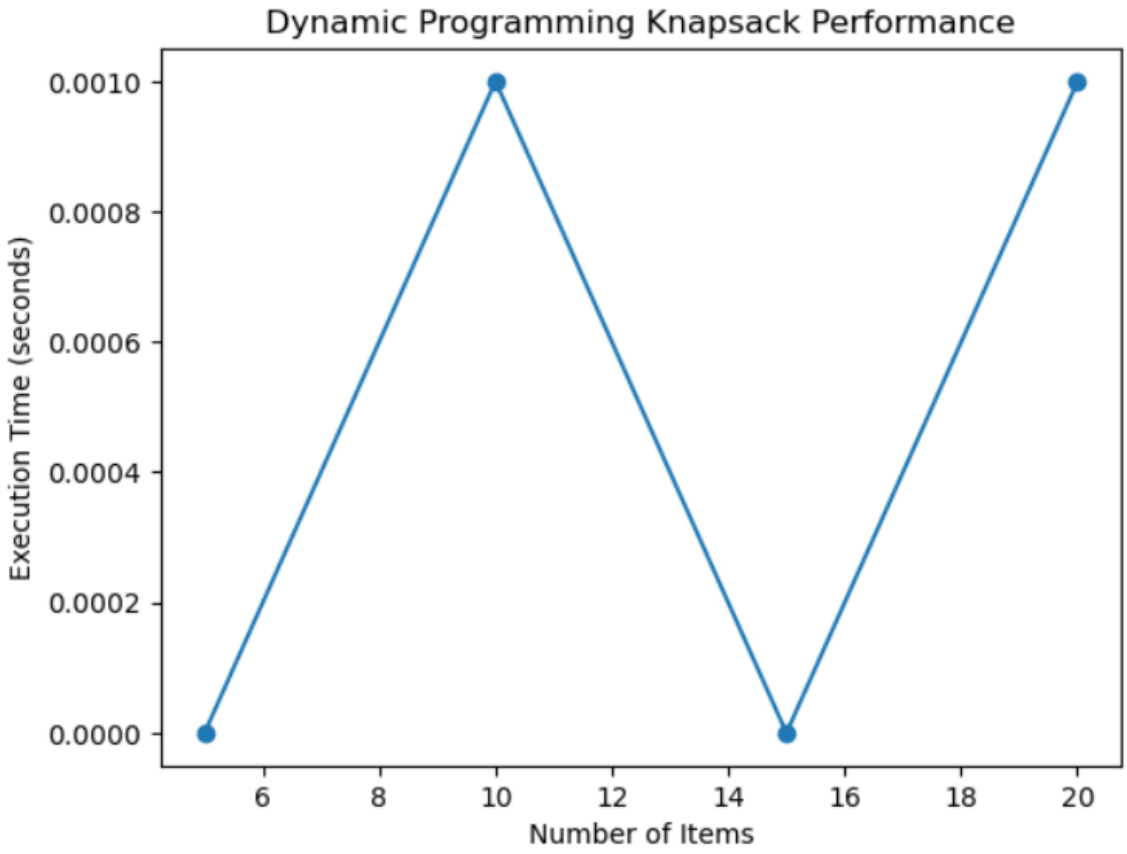
```
import time
import random
import matplotlib.pyplot as plt

def knapsack(W,wt,val,n):
    K = [[0 for x in range(W+1)] for x in range(n+1)]
    for i in range(n+1):
        for w in range(W+1):
            if i==0 or w==0:
                K[i][w]=0
            elif wt[i-1]<=w:
                K[i][w]=max(val[i-1]+K[i-1][w-wt[i-1]],K[i-1][w])
            else:
                K[i][w]=K[i-1][w]
    return K[n][W]

sizes=[5, 10, 15, 20]
times=[]
for n in sizes:
    weights=[random.randint(1,20) for _ in range(n)]
    values=[random.randint(10,100) for _ in range(n)]
    capacity=50
    start=time.time()
    result=knapsack(capacity,weights,values,n)
    end=time.time()
    times.append(end-start)
print("Maximum Profit:", result)
plt.plot(sizes, times, marker='o')
plt.xlabel("Number of Items")
plt.ylabel("Execution Time (seconds)")
plt.title("Dynamic Programming Knapsack Performance")
plt.show()
```

Output :

```
Maximum Profit: 622
```



Observations:

- Dynamic programming stores solutions of sub-problems in a table.
- It avoids repeated calculations and ensures optimal results.
- The complexity depends on the number of items and knapsack capacity.

TASK 5: TRAVELLING SALESMAN PROBLEM (TSP)

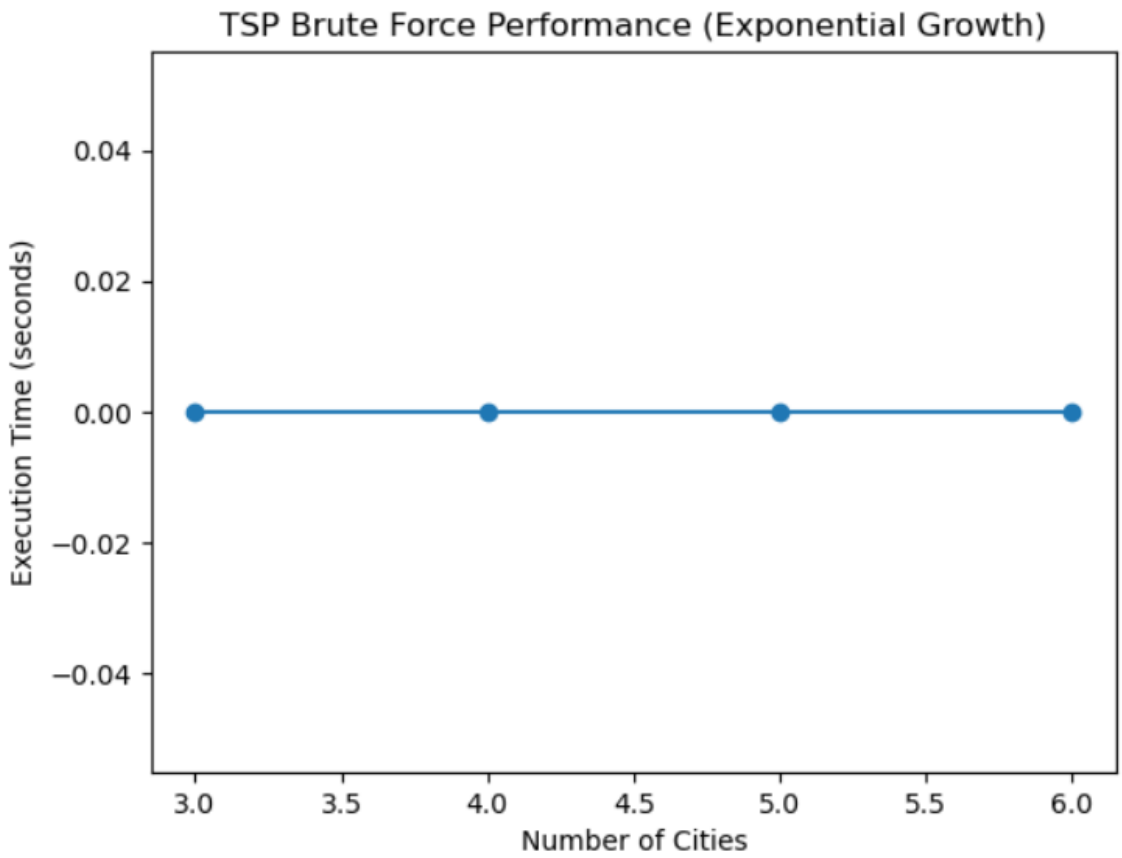
Code :

```
import time
import itertools
import random
import matplotlib.pyplot as plt

def tsp_bruteforce(graph, start):
    n=len(graph)
    vertices=list(range(n))
    vertices.remove(start)
    min_path=float('inf')
    for perm in itertools.permutations(vertices):
        current_cost=0
        k=start
        for j in perm:
            current_cost+=graph[k][j]
            k=j
        current_cost+=graph[k][start]
        min_path=min(min_path, current_cost)
    return min_path

sizes=[3,4,5,6]
times=[]
for n in sizes:
    graph=[[random.randint(1,20) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        graph[i][i]=0
    start=0
    start_time=time.time()
    tsp_bruteforce(graph, start)
    end_time=time.time()
    times.append(end_time-start_time)
plt.plot(sizes,times,marker='o')
plt.xlabel("Number of Cities")
plt.ylabel("Execution Time (seconds)")
plt.title("TSP Brute Force Performance (Exponential Growth)")
plt.show()
```

Output :



Observations:

- The brute force approach checks all possible city permutations.
- The time complexity is O(n!), which grows very quickly.
- This shows why TSP becomes impractical for large number of cities.